

Architecture Microservices

REST



Mouloud Menceur
mouloud.menceur@gmail.com

REST (REpresentational State Transfer)

Style d'architecture définissant un ensemble de contraintes et de propriétés basées sur le protocole **HTTP** - Défini dans la thèse de Roy Fielding en 2000

“Architectural Styles and the Design of Network-based Software Architectures”

Souvent perçu comme une alternative / réponse à la complexité de **SOAP**

SOAP : Au coeur des web services / SOA dans les années 2000

basé sur une communication de type RPC (HTTP + XML)

Multiples critiques :

- **pbs d'interopérabilité** pourtant une de ses raisons d'être
- complexité de la technologie qui **s'est éloignée des principes fondamentaux du Web**

REST (REpresentational State Transfer)

Simplicité de REST (versus SOAP)

REST

La requête

```
GET https://api.{fakecompany}.com/sum?a=40&b=2
```

La réponse

```
42
```

La documentation

```
La ressource « somme » est la somme de deux paramètres entiers  $a$  et  $b$ , représentée par une chaîne de caractères.
```

Source : <https://blog.octo.com/strategie-d-architecture-api/#soapvsrest>

REST (REpresentational State Transfer)

Simplicité de REST (versus SOAP)

SOAP

La requête

```
<?xml version="1.0" encoding="UTF-8" ?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <ns0:additionner
      xmlns:ns0="http://axis.test.com"
      soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <valeur1 xsi:type="xsd:int">40</valeur1>
      <valeur2 xsi:type="xsd:int">2</valeur2>
    </ns0:additionner>
  </soapenv:Body>
</soapenv:Envelope>
```

La réponse

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <ns1:additionnerResponse
      xmlns:ns1="http://axis.test.com"
      soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <additionnerReturn href="#id0" />
    </ns1:additionnerResponse>
    <multiRef xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
      id="id0" soapenc:root="0"
      soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
      xsi:type="xsd:long">42</multiRef>
  </soapenv:Body>
</soapenv:Envelope>
```

La documentation (WSDL)

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions targetNamespace="http://axis.test.com"
  xmlns:apachesoap="http://xml.apache.org/xml-soap"
  xmlns:impl="http://axis.test.com"
  xmlns:intf="http://axis.test.com"
  xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <!--WSDL created by Apache Axis version: 1.3
  Built on Oct 05, 2005 (05:23:37 EDT)-->
  <wsdl:message name="additionnerRequest">
    <wsdl:part name="valeur1" type="xsd:int"/>
    <wsdl:part name="valeur2" type="xsd:int"/>
  </wsdl:message>
  <wsdl:message name="additionnerResponse">
    <wsdl:part name="additionnerReturn" type="xsd:long"/>
  </wsdl:message>
  <wsdl:portType name="Calculer">
    <wsdl:operation name="additionner" parameterOrder="valeur1 valeur2">
      <wsdl:input message="impl:additionnerRequest" name="additionnerRequest"/>
      <wsdl:output message="impl:additionnerResponse" name="additionnerResponse"/>
    </wsdl:operation>
  </wsdl:portType>
  <wsdl:binding name="CalculerSoapBinding" type="impl:Calculer">
    <wsdlsoap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
    <wsdl:operation name="additionner">
      <wsdlsoap:operation soapAction="" />
      <wsdl:input name="additionnerRequest">
        <wsdlsoap:body encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
          namespace="http://axis.test.com" use="encoded"/>
      </wsdl:input>
      <wsdl:output name="additionnerResponse">
        <wsdlsoap:body encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
          namespace="http://axis.test.com" use="encoded"/>
      </wsdl:output>
    </wsdl:operation>
  </wsdl:binding>
  <wsdl:service name="CalculerService">
    <wsdl:port binding="impl:CalculerSoapBinding" name="Calculer">
      <wsdlsoap:address location="http://localhost:8080/TestWS/services/Calculer"/>
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
```

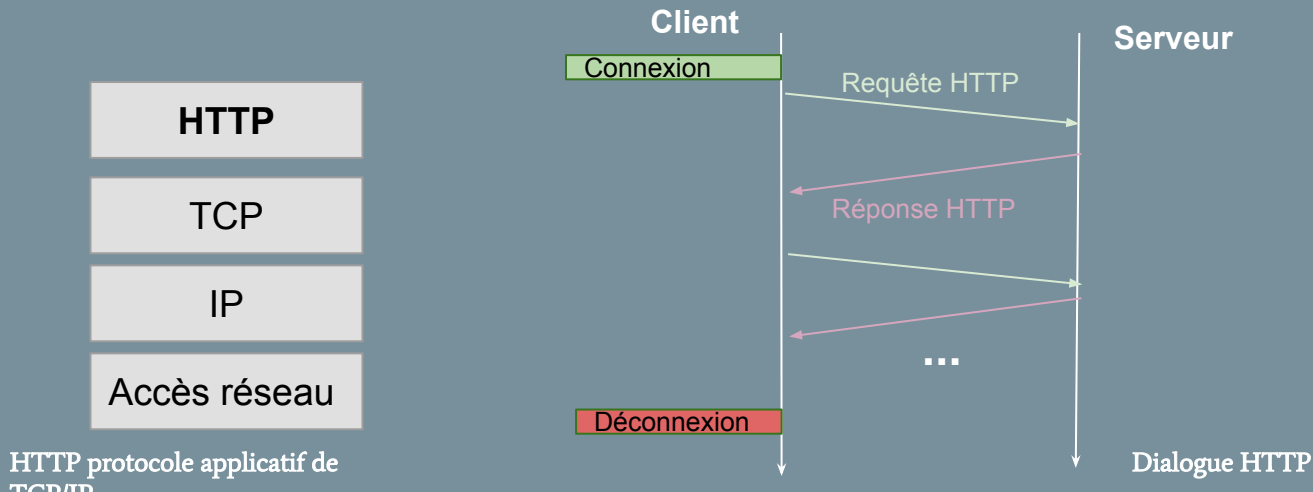
Source : <https://blog.octo.com/strategie-d-architecture-api/#soapvsrest>

Quelques rappels concernant HTTP

Protocole de communication client-serveur développé pour le Web

- HTTP/1.1 : date de 1999, encore présent (25% des échanges)
- **HTTP/2** : le plus utilisé (50%) date de 2014
- HTTP/3 : dernière version majeure datant de 2022 visant à améliorer les performances basé sur le protocole QUIC (UDP au lieu du TCP)

HTTP/2 : protocole applicatif de TCP/IP basé sur des messages textuel de type requête/réponse.



Quelques rappels concernant HTTP

Requêtes HTTP

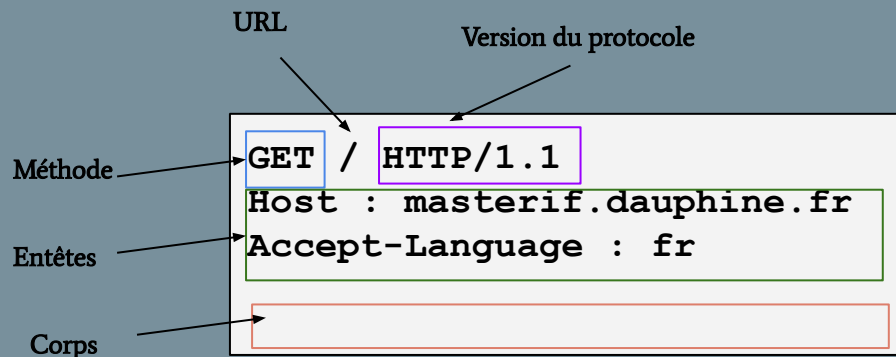
1ère ligne contient :

- la méthode ou commande HTTP (ici GET)
- l'URL de la ressource à partir de la racine du serveur (ici /)
- La version du protocole HTTP (ici 1.1)

Suivi des entêtes optionnels : une entête par ligne ici :

- Host : nom de domaine du site (masterif.dauphine.fr)
- Accept-Language : les langues que le client est capable de comprendre (fr-français)

Suivi du corps du message : (ici vide pour la méthode GET)



Quelques rappels concernant HTTP

Principales Requêtes HTTP

GET : récupère une ressource (page, données) depuis l'URL. Méthode idempotente et qui ne modifie pas la ressource

POST : crée une ressource. Les informations pour créer la ressource sont envoyées dans le corps de la requête. Entraîne en général un changement d'état sur le serveur

PUT : remplace la ressource, indiquée par l'URL, par le corps de la requête. Entraîne en général un changement d'état sur le serveur

DELETE : supprime la ressource indiquée par l'URL. Entraîne en général un changement d'état sur le serveur

HEAD : récupère des informations sur la ressource, sans demander la ressource elle-même. Identique à une requête GET sans les données mais uniquement l'en-tête

Quelques rappels concernant HTTP

Entêtes HTTP

Permettent au client et au serveur de transmettre des informations supplémentaires avec la requête ou la réponse . Un entête de requête est constitué de son nom suivi d'un ':' puis de sa valeur.

Par Ex : (https://en.wikipedia.org/wiki/List_of_HTTP_header_fields)

Host:masterif.dauphine.fr

Indique le nom de domaine du serveur et éventuellement le numéro de port TCP sur lequel le serveur écoute.

Cookie: nom-cookie-1=valeur cookie-1;nom-cookie-2=valeur cookie-2;...

contient les cookies stockés précédemment envoyés par le serveur.

Set-Cookie:nom-cookie-1=valeur cookie-1;nom-cookie-2=valeur cookie-2;...

envoie des cookies du serveur au client.

Content-type: application/x-www-form-urlencoded

Spécifie le type MIME du corps de requête, il sera très utile dans le cas d'une requête POST.

Content-Length: 60

Spécifie la longueur du corps du message HTTP

Quelques rappels concernant HTTP

Corps du message HTTP

Un message HTTP se termine par un corps correspondant au contenu de la requête

- Contenu généralement vide dans les requêtes GET, HEAD, DELETE
- Le corps des messages POST et PUT contiennent les données de la ressource (à créer / modifier)

Les entêtes Content-Type et Content-Length permettent d'indiquer le type MIME et la longueur en octets du corps du message

Quelques rappels concernant HTTP

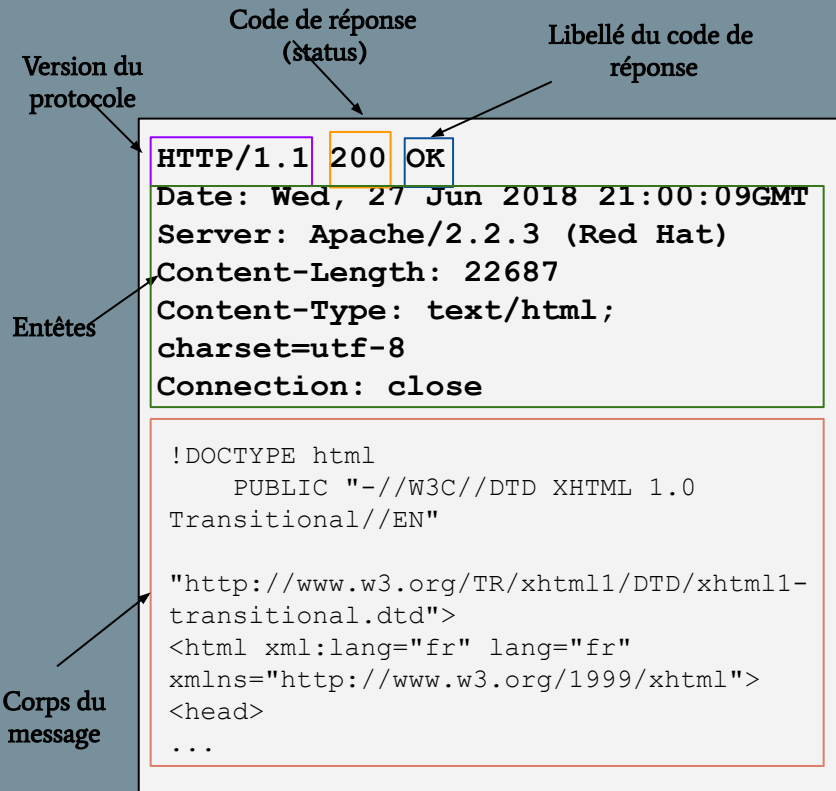
Réponses HTTP

1ère ligne contient :

- La version du protocole HTTP (ici 1.1)
- Le code de réponse (ici 200)
- Le libellé du code de réponse (ici OK)

Suivi des entêtes optionnels : une entête par ligne

Suivi du corps du message : ici le contenu de la page d'accueil (html) de <http://www.masterif.dauphine.fr/>



Quelques rappels concernant HTTP

Codes de statut HTTP

Catégorie	Signification	codes les plus courants
1XX	Réponses informatives	
2XX	Réponses de succès	200 : succès de la requête (GET, POST) 201 : succès de la requête (PUT) : création de la ressource
3XX	Messages de redirection	301 et 302 : redirections permanente et temporaire
4XX	Réponses d'erreur dans la requête du client	401 : authentification nécessaire pour obtenir la ressource 403 : accès à la ressource refusé 404 : ressource non trouvée
5XX	Réponses d'erreur côté serveur	500 : Le serveur a rencontré une situation qu'il ne sait pas traiter 503 : Le serveur n'est pas prêt pour traiter la requête (surchargé, arrêté...) 504 : le serveur n'a pas répondu
https://en.wikipedia.org/wiki/List_of_HTTP_status_codes		

REST (REpresentational State Transfer)

Principes de l'approche REST

Définition d'APIs pour des services web caractérisées par :

- des ressources exposée par des **URI de base** , ex. `http://monapi.miageif.dauphine.fr/resource/`
- Une représentation pour les données par un **type de média** , ex. `application/json`, `application/xml`
- Des messages basés sur des méthodes **HTTP standards** , ex. : GET, POST, PUT, DELETE...
- Des interactions sans état : pas de contexte client entre deux requêtes passées au service

Principes de l'approche REST

1. Architecture basée sur la notion de ressource

Une ressource :

est définie au travers d'une URI (Uniform Resource Identifier)	URI, terme préférée à URL. Permet de localiser une ressource de manière permanente.	Ex : /Devises pour l'URI de la liste des devises
possède un identifiant unique	Une ressource donnée doit avoir un moyen d'être identifiable de manière unique et localisable à travers une URI	Ex : /Devises/EUR URI de la devise EUR (identifiant unique)
A une ou plusieurs représentations permettant de la manipuler	Une ressource est un "objet" abstrait avec plusieurs représentations possibles permettant de manipuler la ressource. Plusieurs représentations sont possibles (format json, XML, pdf, autre...)	Ex : représentation json de la devise EUR : {"currency_code" : "EUR", "label" : "Euro"}

Principes de l'approche REST

2. Interfaces uniformisées

Interactions avec les ressources, de type client-serveur, basées **l'utilisation des méthodes** (ou verbes) et **des codes de statuts HTTP** standard

GET	Lecture / récupération de la ressource sans changement d'état et appels idempotents	Ex : GET /Devises/EUR HTTP/1.1 GET /Devises HTTP/1.1
POST	Création ou mise à jour de la ressource (ou PUT pour la mise à jour)	Ex : POST /Devises HTTP/1.1 Content-Type: application/json Content-Length: 43 {"currency_code" : "EUR", "label" : "Euro"}
DELETE	Suppression de la ressource	Ex : DELETE /Devises/EUR

Principes de l'approche REST

3. Conversation sans état (stateless)

Pas de maintien de session HTTP au niveau du serveur , charge au client de gérer son état

Les messages sont sans état et auto-décrits

chaque requête comporte toutes les informations : l'état est conservé dans les ressources

Les états du serveur sont exposés comme une ressource

Permet une forte scalabilité horizontale

Toutes les instances du service peuvent traiter toutes les requêtes (pas de contexte)

Meilleure résilience

En cas d'erreurs/crashes, démarrer une /des nouvelles instances suffit (pas de contexte)

Principes de l'approche REST

4. Utilisation du cache HTTP

Éviter de requêter systématiquement le serveur d'origine (latence / surcharge)
cela induit

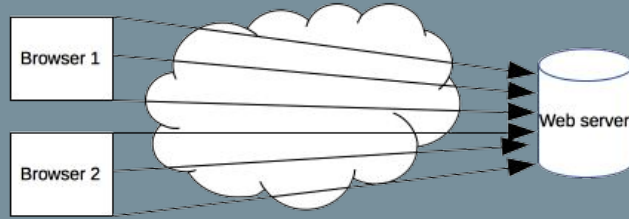
Améliorer les performances perçues côté client

Attention à la durée de validité du cache

2 type de cache :

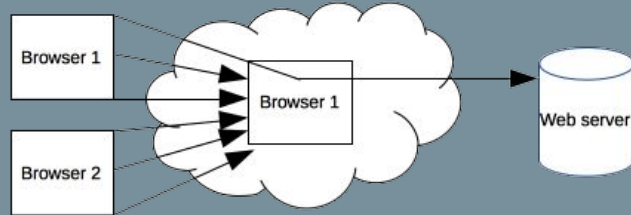
- Au niveau du client (comme sur un navigateur web)
- Sous forme de proxy du serveur d'origine

No cache



All identical requests are going through to the server.

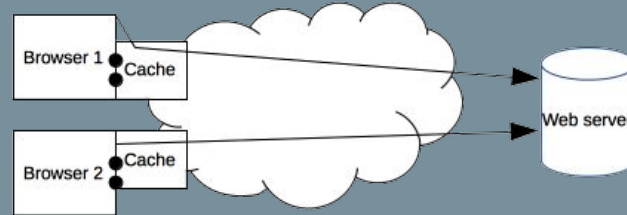
Shared cache



The first request is going through.

Subsequent identical requests are served by the shared cache.
(more efficient)

Local (private) cache



The first request of each client is going through.

Subsequent identical requests are not even sent, but served by the local cache.
(most efficient, except for first requests)

Principes de l'approche REST

5. Systèmes en couches

Principe : lorsqu'un client émet une requête à un service REST, il ne doit pas pouvoir déterminer s'il est connecté directement au serveur final ou à un serveur intermédiaire

Permet d'étendre facilement le système, renforcer la sécurité, améliorer la scalabilité...
(cache, API gateway, load-balancer...)

Modèle de maturité de Richardson

Modèle de référence introduit en 2008

Décompose la mise en oeuvre des principes REST en différents niveaux

Permet également d'évaluer le degré de maturité d'un système REST

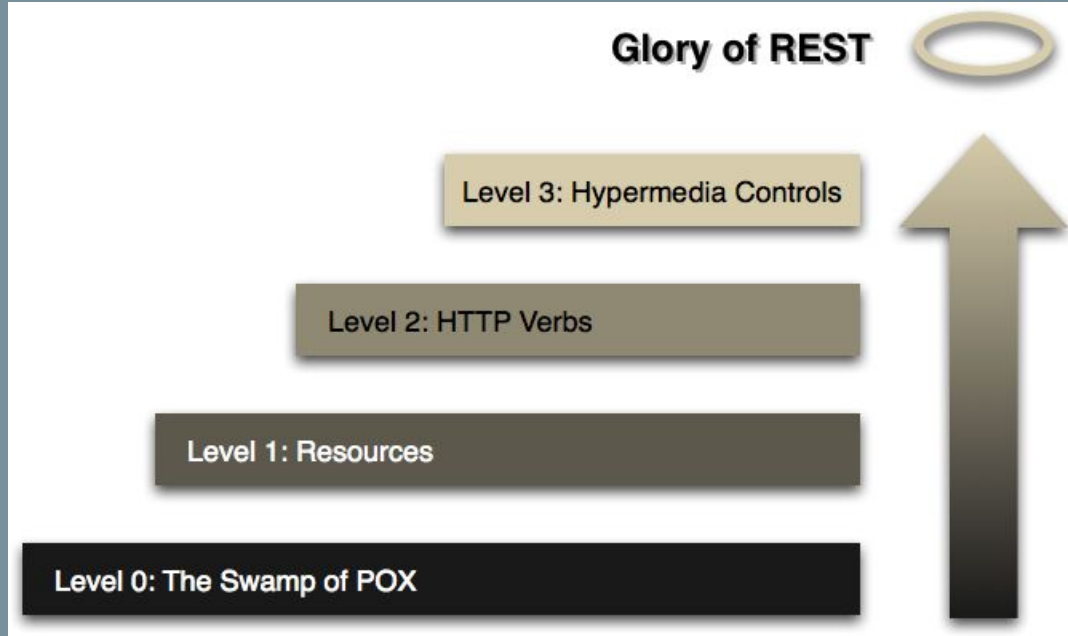
Les web services implémentant les principes REST sont dits **RESTful**



Article de référence de Martin Fowler :

<https://martinfowler.com/articles/richardsonMaturityModel.html>

Modèle de maturité de Richardson



Source :

<https://martinfowler.com/articles/richardsonMaturityModel.html>

Modèle de maturité de Richardson

Niveau 0 - RPC sur HTTP

Point de départ du modèle - pas vraiment REST

HTTP, vu comme un transport, pour communiquer avec un service distant

Principes similaires à SOAP

Toutes les **requêtes** sont envoyées vers la même URI et sont complètement décrites dans un flux XML (POX = Plain Old XML)

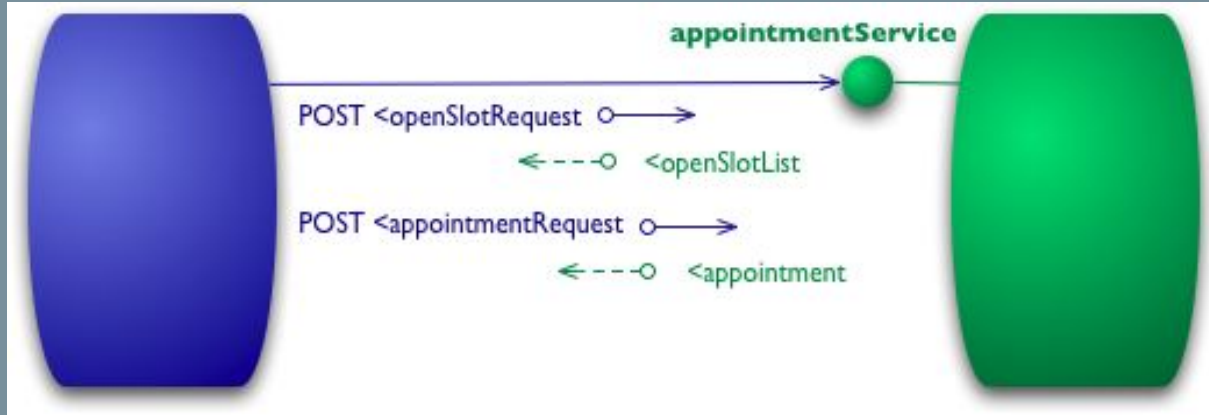
Les appels sont effectués avec un POST

Réponses : complètement décrites le corps du message XML, y compris le statut.

Même code de réponse y compris en cas d'erreur, code HTTP 200 - OK

Modèle de maturité de Richardson - niveau 0

Exemple : service de prise de rdv chez le médecin (M. Fowler)



Requête vers le service **/appointmentService** afin d'obtenir la liste des rdv disponibles, requête (`openSlotRequest`) et paramètres (date et nom du médecin) dans le corps du message

```
POST /appointmentService HTTP/1.1
```

```
<openSlotRequest date = "2018-08-04" doctor = "mjones"/>
```

Modèle de maturité de Richardson - niveau 0

Réponse avec la liste des rdv disponibles

```
HTTP/1.1 200 OK

<openSlotList>
  <slot start = "1400" end = "1445"> <doctor id = "mjones"/></slot>
  <slot start = "1600" end = "1645"> <doctor id = "mjones"/></slot>
</openSlotList>
```

Envoi d'une 2ème requête (appointmentRequest) vers le même service /**appointmentService** afin de réserver un des créneaux de rendez-vous disponibles

```
POST /appointmentService HTTP/1.1

<appointmentRequest>
  <slot doctor = "mjones" start = "1400" end = "1445"/><patient id
  ="jsmith"/>
</appointmentRequest>
```

Modèle de maturité de Richardson - niveau 0

Réponse si la prise de rendez-vous aboutit

```
HTTP/1.1 200 OK
```

```
<appointment>
  <slot doctor = "mjones" start = "1400" end = "1445"/>
  <patient id = "jsmith"/>
</appointment>
```

Réponse si la prise de rendez-vous échoue : corps du message différent mais code de réponse OK

```
HTTP/1.1 200 OK
```

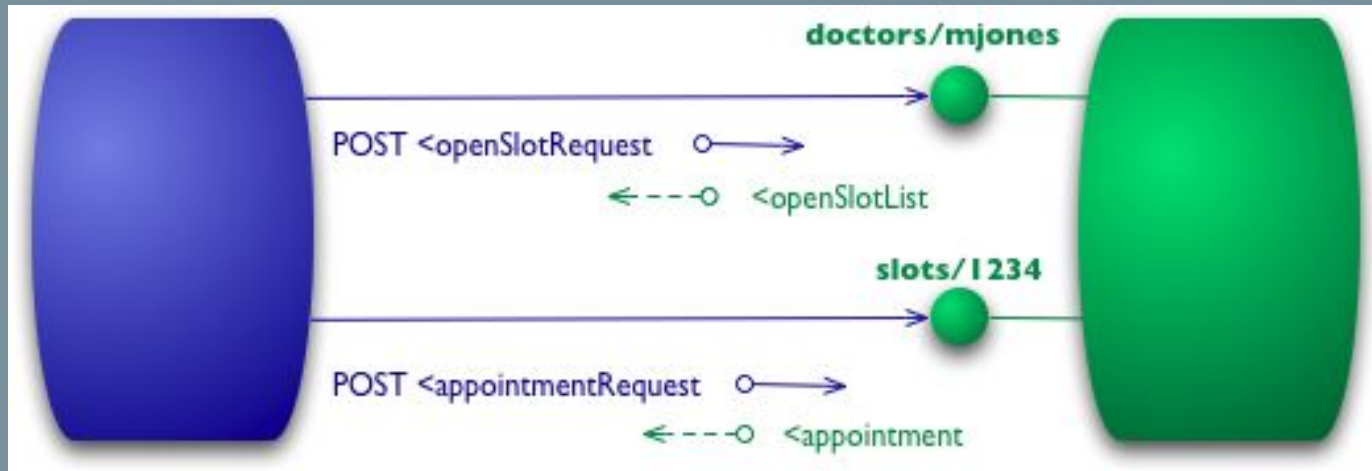
```
<appointmentRequestFailure>
  <slot doctor = "mjones" start = "1400" end = "1445"/>
  <patient id = "jsmith"/>
  <reason>Slot not available</reason>
</appointmentRequestFailure>
```

Modèle de maturité de Richardson

Niveau 1 - Ressources

Introduction de la notion de ressource - Premier pas vers l'utilisation de REST

Les services sont répartis sur plusieurs URI, chaque URI correspond à une ressource



Les appels sont effectués avec un POST et codes de réponse identiques en cas de succès ou d'échec (comme en niveau 0)

Modèle de maturité de Richardson - niveau 1

La 1ère requête, afin d'obtenir la liste des rdv disponibles, est envoyée vers l'URI d'une ressource de type médecin `/doctors/mjones` (ici "mjones"). Le corps du message XML ne porte plus l'information relative au médecin.

```
POST /doctors/mjones HTTP/1.1  
  
<openSlotRequest date = "2018-08-04"/>
```

La réponse contient désormais un identifiant de ressource pour chaque créneau de rdv

```
HTTP/1.1 200 OK  
  
<openSlotList>  
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1445"/>  
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1645"/>  
</openSlotList>
```

Modèle de maturité de Richardson - niveau 1

La 2ème requête afin de réserver un des créneaux de rdv disponibles porte maintenant sur la ressource cible ici `/slots/1234` (ici le créneau de rdv d'id 1234)

```
POST /slots/1234 HTTP/1.1
```

```
<appointmentRequest> <patient id = "jsmith"/>
</appointmentRequest>
```

La réponse est similaire à celle du niveau 0, y compris en cas d'échec (créneau disponible)

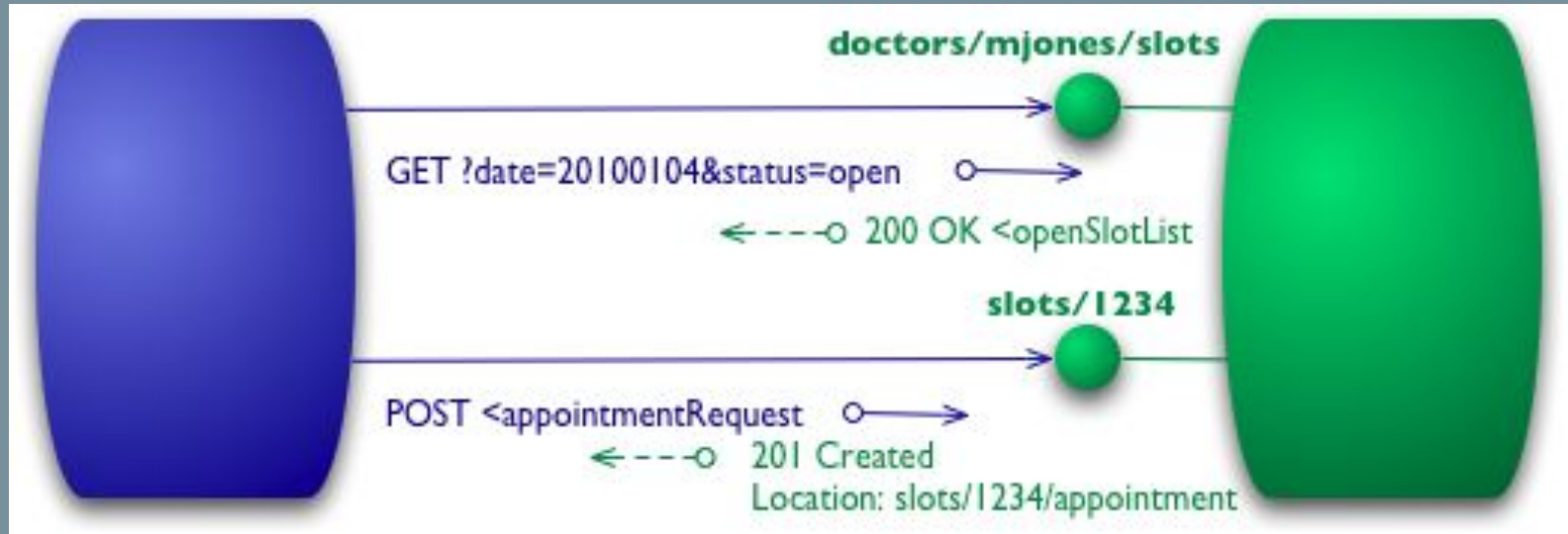
```
HTTP/1.1 200 OK
```

```
<appointment>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1445"/>
  <patient id = "jsmith"/>
</appointment>
```

Modèle de maturité de Richardson

Niveau 2 - verbes et codes de retour HTTP

Tirer parti de HTTP, **protocole applicatif** sous-jacent, en utilisant les verbes HTTP afin d'interagir avec les ressources et les code de retour afin de gérer les statuts des requêtes



Modèle de maturité de Richardson - niveau 2

Utilisation de **GET** pour la 1ère requête afin d'obtenir la liste des rdv disponibles pour un médecin donné

```
GET /doctors/mjones/slots?date=20180804&status=open HTTP/1.1
Host: resadocteur.fr
```

La réponse est identique à celle obtenue avec le POST (niveau 1)

```
HTTP/1.1 200 OK

<openSlotList>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1445"/>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1645"/>
</openSlotList>
```

Modèle de maturité de Richardson - niveau 2

Utilisation de **POST** pour la 2ème requête afin de réserver un des créneaux. L'utilisation du **POST** matérialise un changement d'état sur une ressource (création/mise à jour)

```
POST /slots/1234 HTTP/1.1
```

```
<appointmentRequest> <patient id = "jsmith"/> </appointmentRequest>
```

Les **réponses** sont similaires à celles du niveau 1 à l'exception du **code de retour** (statut HTTP)
- Succès de la requête : code de retour **201** indiquant la création d'une nouvelle ressource, l'URI de cette ressource est indiquée dans le header HTTP Location (ici slots/1234/appointment)

```
HTTP/1.1 201 Created
```

```
Location: slots/1234/appointment
```

```
<appointment> <slot id = "1234" doctor = "mjones" start = "1400" end =  
"1445"/> <patient id = "jsmith"/> </appointment>
```

Modèle de maturité de Richardson - niveau 2

Les **réponses** sont similaires à celles du niveau 1 à l'exception du **code de retour** (statut HTTP) - Echec de la requête : le serveur retourne un code HTTP adapté, correspondant à la raison de l'échec. Par exemple ici le code 409 `Conflict` indiquant un conflit dans la réservation du créneau, le créneau a été réservé entre temps...

“The HTTP 409 Conflict response status code indicates a request conflict with current state of the server.” (<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/409>)

```
HTTP/1.1 409 Conflict
```

```
<openSlotList>
```

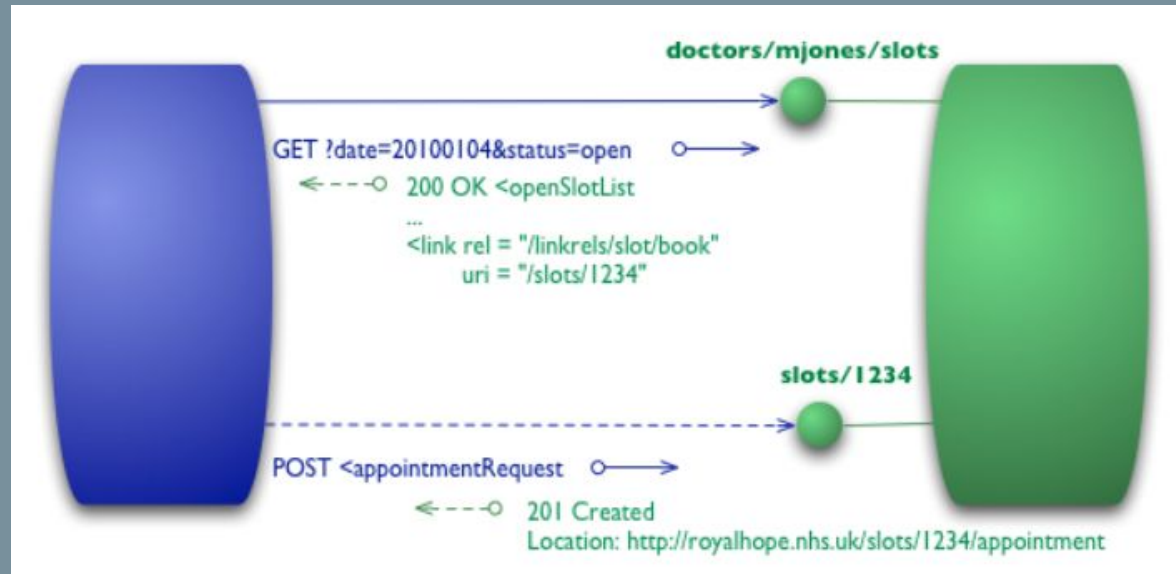
```
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1650"/>
```

```
</openSlotList>
```

Modèle de maturité de Richardson

Niveau 3 - HATEOAS - The glory of REST !

Idée : reprendre le principe de la navigation web *classique* via les liens hypermédia afin de déterminer les transitions possibles depuis une réponse / ressource donnée



Modèle de maturité de Richardson - niveau 3

En réponse à la requête **GET** afin d'obtenir la liste des rdv disponibles pour un médecin donné...

```
GET /doctors/mjones/slots?date=20180804&status=open HTTP/1.1
Host: resadocteur.fr
```

... la réponse, obtenue au niveau 1, est complétée avec l'URI de chaque ressource obtenue (ici les créneaux de rdv)

```
HTTP/1.1 200 OK
```

```
<openSlotList>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1445">
    <link rel = "/linkrels/slot/book" uri = "/slots/1234"/> </slot>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "16445">
    <link rel = "/linkrels/slot/book" uri = "/slots/5678"/> </slot>
</openSlotList>
```

Modèle de maturité de Richardson - niveau 3

En réponse à la requête **POST** afin de réserver un créneau de rdv...

```
POST /slots/1234 HTTP/1.1
<appointmentRequest> <patient id = "jsmith"/> </appointmentRequest>
```

... En cas de succès (code HTTP 201), la réponse contient également les hyperliens associées à la ressource (ici le créneau de rdv /slots/1234)

```
HTTP/1.1 201 Created
Location: http://resadocteur.fr/slots/1234/appointment
<appointment>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
  <link rel = "/linkrels/appointment/cancel" uri = "/slots/1234/appointment"/>
  <link rel = "/linkrels/help" uri = "/help/appointment"/>
  <link rel = "self" uri = "/slots/1234/appointment"/>
  <link rel = "/linkrels/appointment/contact" uri = "/patients/jsmith/contact"/>
</appointment>
```

Modèle de maturité de Richardson - niveau 3

Les bénéfices de HATEOAS

L'API REST est auto-documentée : les hyperliens montrent les “actions” possibles à partir d'une réponse donnée. Même si cette documentation n'est pas complète, elle fournit un premier niveau

cf. `/linkrels/appointment/cancel` et `self` qui portent sur la même URI `/slots/1234/appointment`.

Dans la première URI, il faudra certainement utiliser le verbe DELETE et pour la seconde le verbe GET.

L'API est auto-découvrable : en ajoutant des nouveaux hyperliens, les développeurs de l'API REST peuvent communiquer sur les nouvelles fonctionnalités

Les développeurs de l' API refactoring des URI sans impacter les clients pour peu qu'ils utilisent systématiquement les URI fournies dans les hyperliens.

Modèle de maturité de Richardson - niveau 3

Les critiques d'HATEOAS

La mise en oeuvre est souvent jugée complexe, la découverte de services purement HATEOAS est très séduisante mais peut-être difficile à appréhender

Une partie de cette complexité est déportée du côté client qui doit également être capable de «bien comprendre» la sémantique de l'API

La conception de l'API doit permettre une navigation simple et intelligible

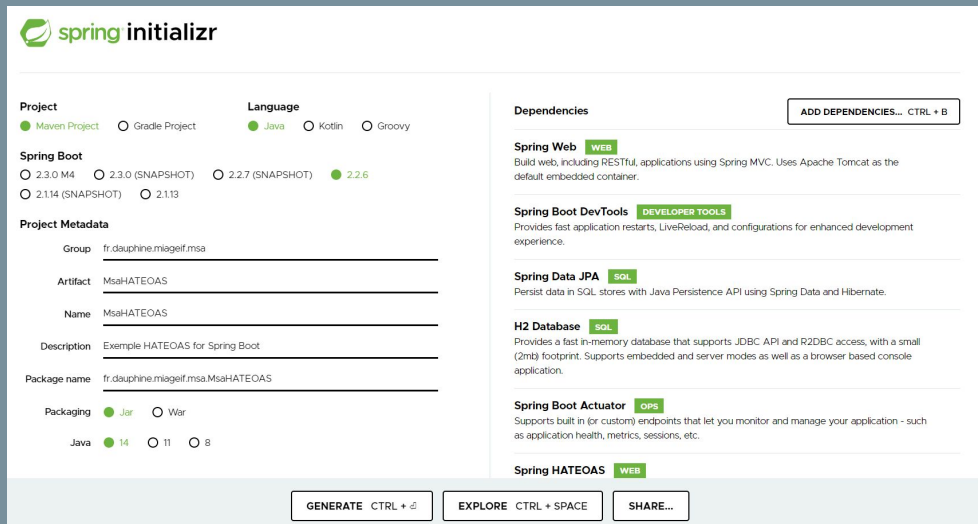
HATEOAS versus Pragmatic REST

Définir une API simple d'utilisation et performante (niveau 2 du modèle de maturité de Richardson)

HATEOAS avec Spring Boot

Ajout de la dépendance Maven HATEOAS

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-hateoas</artifactId>  
</dependency>
```



The screenshot shows the Spring Initializr web application. The 'Project' section has 'Maven Project' selected. The 'Language' section has 'Java' selected. The 'Spring Boot' section has '2.2.6' selected. The 'Project Metadata' section has the following fields: Group (fr.dauphine.miageif.msa), Artifact (MsaHATEOAS), Name (MsaHATEOAS), Description (Exemple HATEOAS for Spring Boot), and Package name (fr.dauphine.miageif.msa.MsaHATEOAS). The 'Packaging' section has 'Jar' selected. The 'Dependencies' section has 'Spring Web' (WEB), 'Spring Boot DevTools' (DEVELOPER TOOLS), 'Spring Data JPA' (SQL), 'H2 Database' (SQL), 'Spring Boot Actuator' (OPS), and 'Spring HATEOAS' (WEB) selected. The 'ADD DEPENDENCIES... CTRL + B' button is visible. At the bottom, there are buttons for 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'.

HATEOAS avec Spring Boot

Refactoring de l'exemple vu dans le cours “Développement avec Spring Boot”

Seul le controleur `REST ChangeController` est modifié (`@RestController`) afin de fournir une API RestFul HATEOAS

Un Service REST `tous-tauxchange` est ajouté, il permet de récupérer l'ensemble des taux de change

← → ↻ ⓘ localhost:8001/devise-change-hateoas/tous-tauxchange

```
[{"id":10001,"source":"EUR","dest":"USD","taux":1.17,"port":0},
{"id":10002,"source":"EUR","dest":"JPY","taux":130.00,"port":0},
{"id":10003,"source":"USD","dest":"GBP","taux":0.75,"port":0}]
```

HATEOAS avec Spring Boot

Refactoring de l'exemple vu dans le cours “Développement avec Spring Boot”)

Le service REST `/devise-change-hateoas/source/{source}/dest/{dest}` permettant de récupérer un taux de change est modifié afin d'enrichir la réponse avec un hyperlien vers la ressource **tous-tauxchange**



A screenshot of a web browser window. The address bar shows the URL `localhost:8001/devise-change-hateoas/source/EUR/dest/USD`. The page content displays a JSON response with a highlighted yellow background. The JSON object includes an `_links` property with a single entry for `tous-tauxchange`, providing a `href` to the root of the API.

```
{ "id": 10001, "source": "EUR", "dest": "USD", "taux": 1.17, "port": 0, "_links": { "/devise-change-hateoas/tous-tauxchange": { "href": "http://localhost:8001/devise-change-hateoas/tous-tauxchange" } } }
```

Dév. avec Spring Boot

```
...
@RequestMapping(value =
"/devise-change-hateoas/source/{source}/dest/{dest}", method =
RequestMethod.GET, produces = "application/json; charset=UTF-8")
    public Resource<TauxChange> retrouveTauxChange
        (@PathVariable String source, @PathVariable String dest){
        TauxChange tauxChange = repository.findBySourceAndDest(source, dest);
        /* on enveloppe l'objet du domaine, ici l'entité TauxChange, dans une ressource
HATEOAS (org.springframework.hateoas.Resource) */
        Resource<TauxChange > resource = new Resource<TauxChange>(tauxChange);
        /* on construit le hyperlien HATEOAS associé au service REST de
récupération de l'ensemble des taux de change*/
        ControllerLinkBuilder linkTo =
linkTo(methodOn(this.getClass()).tousTauxChange());
        /* On ajoute ce lien à la ressource attachée au TauxChange en précisant
l'URI */
        resource.add(linkTo.withRel("/devise-change-hateoas/tous-tauxchange"));
```

Dév. avec Spring Boot

```
...  
// GET /devise-change-hateoas/tous-tauxchange  
retourne l'ensemble des taux de change  
@GetMapping("/devise-change-hateoas/tous-tauxchange")  
    public List<TauxChange> tousTauxChange() {  
        // méthode de JpaRepository : récupère toutes les entités  
        return repository.findAll();  
    }
```

ChangeController.java